ML Problem Solving: Convolutions From Scratch and Pytorch CNNs

Overview

In this problem-solving we will be implementing convolution operations from scratch, perform pre-processing of a simple image dataset, using Pytorch's rich API for defining, training and performing inference of CNN-based Neural Networks, and perform hyper-parameter tuning using different Pytorch optimizers.

Problem Breakdown

Module	Topic	Target Dataset / Inputs
Problem 1	Discrete Convolutions & Layer Output Shapes	Synthetic 2D Arrays & Tensors
Problem 2	1D and 2D Convolutions from Scratch	Synthetic 1D and 2D Tensors
Problem 3	Fashion-MNIST Pipeline, Preprocessing & Splits	Fashion-MNIST Dataset
Problem 4	Built-in PyTorch Convolutions & Architectural Variations	Fashion-MNIST Dataset
Problem 5	Activations, Regularization & Comparative Evaluation	Fashion-MNIST Dataset

Problem 1: Discrete Convolutions and Layer Shape Analysis

1.1 Conceptual Background

Discrete Cross-Correlation vs. Convolution In standard machine learning frameworks (including PyTorch), the operation termed "convolution" is mathematically implemented as discrete cross-correlation. The spatial kernel is slid across the input domain without spatial reflection:

$$C(x, y) = \sum_m \sum_n I(x + m, y + n) \cdot K(m, n)$$

Here I denotes the input 2D image matrix, K denotes the 2D filter kernel and Coordinates (m, n) index the spatial positions of the kernel. When the anchor (center) of kernel K is set to its top-left pixel, the relative coordinates range over $m \in \{0, \ldots, K_h - 1\}$ and $n \in \{0, \ldots, K_w - 1\}$.

Padding and Output Dimension Dynamics

- **Valid Padding ($P = 0$):** Receptive fields must lie strictly inside the boundaries of I. No artificial boundary values are introduced. The spatial dimensions shrink according to kernel size and stride.
- **Same Padding:** Zeros are padded along the boundaries such that, when stride $S = 1$, the spatial height and width of the output match the input spatial dimensions exactly ($H_{out} = H_{in}, W_{out} = W_{in}$).
- **General Dimension Formula:** Given input height H_{in}, kernel size K_h, padding P_h, dilation D_h, and stride S_h: $H_{out} = \left\lfloor \frac{H_{in}+2P_h-D_h(K_h-1)-1}{S_h} \right\rfloor + 1$

1.2 Problem Statement

Consider a single grayscale image I of shape 4×4 and a 2×2 kernel K:

$$I = \begin{bmatrix} 255 & 128 & 128 & 0 \\ 51 & 255 & 0 & 128 \\ 51 & 128 & 128 & 0 \\ 0 & 128 & 0 & 0 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 2 \\ 0 & 2 \end{bmatrix}$$

Centering of K relative to I assumes the **top-left pixel** of the kernel is its anchor.

1. **Pixel Normalization:** Normalize the pixel intensities of I from the unsigned integer range $[0, 255]$ into floating-point numbers in the range $[0.0, 1.0]$.
2. **Same Padding with Stride 1:** Compute the exact output matrix resulting from the 2D discrete convolution of normalized I and K under `padding='same'` and stride $S = 1$.
3. **Valid Padding with Stride 1:** Compute the exact output matrix resulting from the 2D discrete convolution of normalized I and K under `padding='valid'` and stride $S = 1$.
4. **Valid Padding with Stride 2:** Compute the exact output matrix resulting from the 2D discrete convolution of normalized I and K under `padding='valid'` and stride $S = 2$.
5. **Tensor Shape Tracking:** Given an input minibatch tensor x with shape `[4, 1, 28, 28]` (representing 4 single-channel images of size 28×28), compute the output tensor shape after each consecutive layer:

   Layer 1: `nn.Conv2d(in_channels=1, out_channels=16, kernel_size=3, padding=0, stride=1)`; Layer 2: `nn.ReLU()`; Layer 3: `nn.Conv2d(in_channels=16, out_channels=64, kernel_size=3, padding=0, stride=2)`; Layer 4: `nn.ReLU()`; Layer 5: `nn.MaxPool2d(kernel_size=2, stride=2)`; Layer 6: `nn.Dropout(p=0.5)`

1.3 Expectations

- Show step-by-step arithmetic (fractions or 4-decimal rounded values) for each receptive field calculation in Tasks 1 through 4.
- For Task 5, write out the explicit numerical substitution into the spatial output dimension formula for each parametric or downsampling layer.
- Report all tensor shapes using PyTorch four-dimensional format: `[Batch, Channels, Height, Width]`.

Problem 2: 1D and 2D Convolutions from Scratch

2.1 Conceptual Background

Multi-Channel Convolutions In PyTorch, convolutional layers operate across batches of multi-channel representations:

- Input: $X \in \mathbb{R}^{B \times C_{in} \times H_{in} \times W_{in}}$
- Kernels: $W \in \mathbb{R}^{C_{out} \times C_{in} \times K_h \times K_w}$
- Biases: $b \in \mathbb{R}^{C_{out}}$

For each sample b in the batch and output filter c_{out} at position (i, j):

$$\text{Out}(b, c_{out}, i, j) = b(c_{out}) + \sum_{c_{in}=0}^{C_{in}-1} \sum_{m=0}^{K_h-1} \sum_{n=0}^{K_w-1} X\left(b, c_{in}, i \cdot S_h + m, j \cdot S_w + n\right) \cdot W(c_{out}, c_{in}, m, n)$$

1D Convolutions For sequence inputs $X \in \mathbb{R}^{B \times C_{in} \times L_{in}}$ and kernels $W \in \mathbb{R}^{C_{out} \times C_{in} \times K_L}$:

$$\text{Out}(b, c_{out}, i) = b(c_{out}) + \sum_{c_{in}=0}^{C_{in}-1} \sum_{m=0}^{K_L-1} X\left(b, c_{in}, i \cdot S + m\right) \cdot W(c_{out}, c_{in}, m)$$

2.2 Problem Statement

1. **1D Convolution from Scratch:** Implement a pure PyTorch function:

   ```
   def conv1d_scratch(x, weight, bias=None, stride=1, padding=0):
       ...
   ```

   Do not use `torch.nn.Conv1d` or `torch.nn.functional.conv1d`.

2. **2D Convolution from Scratch:** Implement a pure PyTorch function:

   ```
   def conv2d_scratch(x, weight, bias=None, stride=1, padding=0):
       ...
   ```

   Do not use `torch.nn.Conv2d` or `torch.nn.functional.conv2d`.

3. **Equivalence Verification:** Create synthetic test tensors with multiple input/output channels, non-zero padding ($P \geq 1$), and downsampling stride ($S \geq 2$). Calculate the maximum absolute error against official PyTorch functional primitives:

$$\text{diff}_{max} = \max |Y_{scratch} - Y_{torch}|$$

2.3 Expectations

- Support arbitrary batch sizes, input channels, output channels, kernel dimensions, padding values, and strides.
- Ensure optional bias addition is broadcast correctly across all spatial dimensions.
- Explicitly demonstrate that `diff_max < 1e-5` for both 1D and 2D functions.

Problem 3: Fashion-MNIST Acquisition, Pipeline Preprocessing, and Splitting

3.1 Conceptual Background

Preventing Data Leakage In supervised visual classification pipelines:

- **Training Set:** Used for gradient descent optimization.
- **Validation Set:** Used for tuning hyperparameters (learning rate, weight decay, architecture) and early stopping.
- **Test Set:** Held out strictly for final generalization scoring.

Normalization statistics must be sourced either from published population parameters or derived exclusively from the training split.

Fashion-MNIST Specifications

- 70,000 grayscale images (28×28 pixels) across 10 clothing classes.
- Standard distribution: 60,000 train images, 10,000 test images.
- Published population parameters: mean $\mu = 0.2860$, standard deviation $\sigma = 0.3530$.

3.2 Problem Statement

Write an end-to-end data acquisition and ingestion pipeline using `torchvision`:

1. **Transform Definition:** Construct a `transforms.Compose` pipeline incorporating:
   - Conversion to PyTorch tensors (`transforms.ToTensor()`).
   - Channel normalization using $\mu = 0.2860$ and $\sigma = 0.3530$.
2. **Dataset Acquisition:** Download the official `FashionMNIST` training and test sets.
3. **Validation Partitioning:** Using `torch.utils.data.random_split`, partition the 60,000-sample training dataset into:
   - 50,000 samples for model parameter training.
   - 10,000 samples for validation.
   - Fix the split using a seeded `torch.Generator(seed=42)`.
4. **DataLoaders:** Instantiate PyTorch `DataLoader` objects with:
   - Batch size: 64.
   - Training set shuffled (`shuffle=True`); validation and test sets unshuffled (`shuffle=False`).
   - Multi-process loading with `pin_memory=True` when acceleration hardware is present.
5. **Sanity Verification:** Pull a single batch from the training loader and log:
   - Batch feature tensor shape.
   - Batch target tensor shape.
   - Empirical mean and standard deviation of the batch.

3.3 Expectations

- Random partition allocation must be deterministic and fully reproducible across runs.
- Do not apply transforms in a manner that re-normalizes cached tensors repeatedly.
- Output the verified batch dimensions and total batch counts for train, validation, and test loaders.

Problem 4: Built-in PyTorch Convolutions and Architectural Variations

4.1 Conceptual Background

Fully Connected vs. Convolutional Representations

- **Fully Connected (Dense) Layers:** Treat input dimensions independently by flattening matrices into vectors ($28 \times 28 \to 784$). This discards 2D spatial adjacency and requires a large number of parameters ($784 \times H$).
- **Convolutional Layers:** Enforce weight sharing across the spatial grid via sliding kernels. This guarantees translational equivariance and reduces total parameter count.

Structural Constraint All architectures developed in this section are restricted to **at most 3 parametric (learnable) layers**. Pooling, flattening, and non-linear activations do not contain learnable parameters and are excluded from the layer-count ceiling.

4.2 Problem Statement

Implement three distinct PyTorch architectures adhering to the at-most-3-parametric-layer constraint:

1. **Architecture 1: Fully Connected Baseline (MLP)**
   - Input: $1 \times 28 \times 28 \to$ Flatten to 784
   - Parametric Layer 1: `nn.Linear(784, 256)` followed by `nn.ReLU()`
   - Parametric Layer 2: `nn.Linear(256, 64)` followed by `nn.ReLU()`
   - Parametric Layer 3: `nn.Linear(64, 10)`
2. **Architecture 2: Pure Convolutional Network**
   - Parametric Layer 1: `nn.Conv2d(1, 16, kernel_size=3, padding=1)` + `nn.ReLU()` + `nn.MaxPool2d(2, 2)`
   - Parametric Layer 2: `nn.Conv2d(16, 32, kernel_size=3, padding=1)` + `nn.ReLU()` + `nn.MaxPool2d(2, 2)`
   - Parametric Layer 3: `nn.Conv2d(32, 10, kernel_size=3, padding=1)` + `nn.AdaptiveAvgPool2d((1, 1))` + Flatten to `[B, 10]`
3. **Architecture 3: Hybrid ConvNet + Dense Classifier**
   - Parametric Layer 1: `nn.Conv2d(1, 32, kernel_size=3, padding=1)` + `nn.ReLU()` + `nn.MaxPool2d(2, 2)`
   - Parametric Layer 2: `nn.Conv2d(32, 64, kernel_size=3, padding=1)` + `nn.ReLU()` + `nn.MaxPool2d(2, 2)`
   - Flatten spatial grid ($64 \times 7 \times 7 = 3136$)
   - Parametric Layer 3: `nn.Linear(3136, 10)`
4. **Parameter Calculation:** Derive the theoretical parameter count for each layer (weights and biases), and confirm the total programmatically using `sum(p.numel() for p in model.parameters() if p.requires_grad)`.

4.3 Expectations

- Encapsulate each model as an independent `nn.Module` class.
- Verify that a dummy input of shape `[64, 1, 28, 28]` produces output logits of shape `[64, 10]`.
- Provide a clear markdown table comparing the three architectures in terms of layer types, output dimensions, and total parameter counts.

Problem 5: Activations, Regularization, and Comparative Model Evaluation

5.1 Conceptual Background

Non-Linear Activations

- **Rectified Linear Unit (ReLU):** $\text{ReLU}(x) = \max(0, x)$. Highly efficient, but units can become inactive if pre-activations remain negative across the training distribution.
- **Leaky ReLU:** $\text{LeakyReLU}(x) = \max(\alpha x, x)$ with slope $\alpha > 0$ (e.g., 0.05). Maintains a small, non-zero gradient for negative values to prevent dead neurons.

Regularization via Inverted Dropout `nn.Dropout(p)` randomly sets activations to zero during training with probability p. Surviving activations are scaled by $\frac{1}{1-p}$ so that expected layer outputs remain consistent between training and evaluation. Dropout is disabled during `model.eval()`.

5.2 Problem Statement

Using Architecture 3 (Hybrid ConvNet) as a common backbone, implement a modular training and evaluation suite to compare four configurations over 5 training epochs:

1. **Modular Engine:** Implement `train_one_epoch` and `evaluate` functions computing Cross-Entropy Loss and accuracy percentage.
2. **Experimental Variations:**
   - **Model Variant A:** Base Hybrid (`ReLU`, Dropout $p = 0.0$)
   - **Model Variant B:** Non-linear Alternative (`LeakyReLU(negative_slope=0.05)`, Dropout $p = 0.0$)
   - **Model Variant C:** Regularized Base (`ReLU`, Dropout $p = 0.30$ inserted before the linear layer)
   - **Model Variant D:** Regularized Alternative (`LeakyReLU(0.05)`, Dropout $p = 0.30$)
3. **Standard Hyperparameters:**
   - Optimizer: Adam (`lr=1e-3`)
   - Loss function: `nn.CrossEntropyLoss()`
   - Epochs: 5
4. **Generalization Evaluation:** Evaluate all four trained models on the unseen 10,000-sample test set.
5. **Comparative Analysis:** Construct a results table tracking Validation Loss, Validation Accuracy, Test Accuracy, and the Train-to-Validation generalization gap.

5.3 Expectations

- Explicitly toggle training and inference modes via `model.train()` and `model.eval()`.
- Wrap all evaluation passes inside `torch.no_grad()` blocks.
- Provide a written analysis explaining how activation choices and dropout affect the generalization gap.